1. Implement a push function. Start with an array size of 2. If a student pushes an item into a full stack, create a new array that is double the size. Copy all old items over.

```
#include <iostream>

class DynamicStack {
private:
    int* arr;
    int capacity;
    int top;

public:
    DynamicStack() {
        capacity = 2;
        arr = new int[capacity];
        top = -1;
    }

    ~DynamicStack() {
        delete[] arr;
    }

    void push(int value) {
        if (top + 1 == capacity) {
            int newCapacity = capacity * 2;
            int* newArr = new int[newCapacity];

            for (int i = 0; i <= top; ++i) {
                newArr[i] = arr[i];
            }

            delete[] arr;
            arr = newArr;
            capacity = newCapacity;

            std::cout << "Stack full. Resized capacity to " << capacity << "\n";
        }

        arr[++top] = value;
        std::cout << "Pushed " << value << "\n";
    }

    void display() const {
        std::cout << "Stack elements: ";
        for (int i = 0; i <= top; ++i) {
            std::cout << arr[i] << " ";
        }
        std::cout << "\n";
    }
};

int main() {
    DynamicStack stack;
```

```
    stack.push(10);
    stack.push(20);
    stack.push(30);
    stack.push(40);
    stack.push(50);

    stack.display();

    return 0;
}
```

Output:

```
PS C:\Users\HP\Downloads\New folder (2)> cd "c:\Users\HP\Downloads\New folder (2)\" ; if ($?) { g++ 6.cpp -o 6 } ; if ($?) { .\6 }
Pushed 10
Pushed 20
Stack full. Resized capacity to 4
Pushed 30
Pushed 40
Stack full. Resized capacity to 8
Pushed 50
Stack elements: 10 20 30 40 50
```

2. Track the exact number of operations. For a normal push, the cost is 1. For a resizing push, the cost is 1 plus the number of copied items.

```
#include <iostream>

class DynamicStack {
private:
    int* arr;
    int capacity;
    int top;
    int totalOps;

public:
    DynamicStack() {
        capacity = 2;
        arr = new int[capacity];
        top = -1;
        totalOps = 0;
    }

    ~DynamicStack() {
        delete[] arr;
    }

    void push(int value) {
        int opCost = 1; // Base cost for insertion

        if (top + 1 == capacity) {
            int copiedItems = top + 1;
            opCost += copiedItems; // Add cost of copying existing items
```

```
            int newCapacity = capacity * 2;
            int* newArr = new int[newCapacity];

            for (int i = 0; i <= top; ++i) {
                newArr[i] = arr[i];
            }

            delete[] arr;
            arr = newArr;
            capacity = newCapacity;
        }

        arr[++top] = value;
        totalOps += opCost;

        std::cout << "Pushed: " << value
                  << " | Cost: " << opCost
                  << " | Total Ops: " << totalOps
                  << " | Capacity: " << capacity << "\n";
    }

    int getTotalOps() const {
        return totalOps;
    }
};

int main() {
    DynamicStack stack;

    // Push sequence to demonstrate cost tracking
    for (int i = 1; i <= 6; ++i) {
        stack.push(i * 10);
    }

    return 0;
}
```

Output:

```
PS C:\Users\HP\Downloads\New folder (2)> cd "c:\Users\HP\Downloads\New folder (2)\" ; if ($?) { g++ 7.cpp -o 7 } ; if ($?) { .\7 }
Pushed: 10 | Cost: 1 | Total Ops: 1 | Capacity: 2
Pushed: 20 | Cost: 1 | Total Ops: 2 | Capacity: 2
Pushed: 30 | Cost: 3 | Total Ops: 5 | Capacity: 4
Pushed: 40 | Cost: 1 | Total Ops: 6 | Capacity: 4
Pushed: 50 | Cost: 5 | Total Ops: 11 | Capacity: 8
Pushed: 60 | Cost: 1 | Total Ops: 12 | Capacity: 8
```

3. Push 1,000 items in a loop. Print out the total cost and the average cost per push at the very end.

```cpp
#include <iostream>

class DynamicStack {
private:
    int* arr;
    int capacity;
    int top;
    long long totalOps;

public:
    DynamicStack() {
        capacity = 2;
        arr = new int[capacity];
        top = -1;
        totalOps = 0;
    }

    ~DynamicStack() {
        delete[] arr;
    }

    void push(int value) {
        int opCost = 1;

        if (top + 1 == capacity) {
            int copiedItems = top + 1;
            opCost += copiedItems;

            int newCapacity = capacity * 2;
            int* newArr = new int[newCapacity];

            for (int i = 0; i <= top; ++i) {
                newArr[i] = arr[i];
            }

            delete[] arr;
            arr = newArr;
            capacity = newCapacity;
        }

        arr[++top] = value;
        totalOps += opCost;
    }

    long long getTotalOps() const {
        return totalOps;
    }
};

int main() {
    DynamicStack stack;
    const int numPushes = 1000;
```

```
    for (int i = 1; i <= numPushes; ++i) {
        stack.push(i);
    }

    long long totalCost = stack.getTotalOps();
    double averageCost = static_cast<double>(totalCost) / numPushes;

    std::cout << "Total Pushes: " << numPushes << "\n";
    std::cout << "Total Cost (Operations): " << totalCost << "\n";
    std::cout << "Average Cost per Push: " << averageCost << "\n";

    return 0;
}
```

Output:

```
PS C:\Users\HP\Downloads\New folder (2)> cd "c:\Users\HP\Downloads\New folder (2)\" ; if ($?) { g++ 8.cpp -o 8 } ; if ($?) { .\8 }
Total Pushes: 1000
Total Cost (Operations): 2022
Average Cost per Push: 2.022
```

4. Change the resizing rule. Instead of doubling, increase the size by adding +10 slots each time it fills up. Run 1,000 pushes again. Compare the average costs of both methods.

```
#include <iostream>

class DynamicStackPlus10 {
private:
    int* arr;
    int capacity;
    int top;
    long long totalOps;

public:
    DynamicStackPlus10() {
        capacity = 2;
        arr = new int[capacity];
        top = -1;
        totalOps = 0;
    }

    ~DynamicStackPlus10() {
        delete[] arr;
    }

    void push(int value) {
        int opCost = 1;

        if (top + 1 == capacity) {
            int copiedItems = top + 1;
            opCost += copiedItems;
```

```
            int newCapacity = capacity + 10; // Resizing rule: +10 slots
            int* newArr = new int[newCapacity];

            for (int i = 0; i <= top; ++i) {
                newArr[i] = arr[i];
            }

            delete[] arr;
            arr = newArr;
            capacity = newCapacity;
        }

        arr[++top] = value;
        totalOps += opCost;
    }

    long long getTotalOps() const {
        return totalOps;
    }
};

int main() {
    DynamicStackPlus10 stack;
    const int numPushes = 1000;

    for (int i = 1; i <= numPushes; ++i) {
        stack.push(i);
    }

    long long totalCost = stack.getTotalOps();
    double averageCost = static_cast<double>(totalCost) / numPushes;

    std::cout << "Total Pushes: " << numPushes << "\n";
    std::cout << "Total Cost (Operations): " << totalCost << "\n";
    std::cout << "Average Cost per Push: " << averageCost << "\n";

    return 0;
}
```

Output:

```
PS C:\Users\HP\Downloads\New folder (2)> cd "c:\Users\HP\Downloads\New folder (2)\" ; if ($?) { g++ 9.cpp -o 9 } ; if ($?) { .\9 }
Total Pushes: 1000
Total Cost (Operations): 50700
Average Cost per Push: 50.7
```